Name of Course	**Bachelor of Computer Application (BCA)**
Semester	**VI**
Name of Subject	**Digital Image Processing**
Subject Code	**BCA-604-B**
Marks	**75**
Lectures	**50**

Unit I

1. Introduction to MATLAB

- **1.1 Introduction**
- **1.2 Advantages and Disadvantages of MATLAB**
- **1.3 MATLAB Environment**
- **1.4 Using MATLAB Scratch Pad**
- **1.5 Variables and Arrays**
- **1.6 Multidimensional Arrays**
- **1.7 Scalar and Array Operations**

1.1 Introduction to MATLAB:

MATLAB (Matrix Laboratory) is a high-level programming environment widely used in engineering, science, and research for numerical computation, data analysis, and visualization. It provides an easy-to-use interface with powerful built-in functions that help users perform complex mathematical operations efficiently. MATLAB is especially designed for working with matrices, signals, images, and simulation-based tasks.

Some important points about MATLAB are:

1. **High-Level Programming Language:**
   MATLAB allows users to write programs using simple and readable commands, making it suitable for both beginners and advanced users.

2. **Numerical and Scientific Computing:**
   It is mainly used for solving mathematical problems, simulations, algorithm development, and performing large-scale numerical calculations.

3. **Rich Visualization Tools:**
   MATLAB provides excellent 2D and 3D plotting tools that help in visualizing data, graphs, functions, and simulation results easily.

4. **Extensive Toolboxes:**
   MATLAB offers specialized toolboxes for areas like signal processing, control systems, image processing, machine learning, and communication systems, extending its functionality.

5. **Platform Independent:**
   MATLAB programs can run on different operating systems like Windows, Linux, and macOS without modification, making it portable and easy to use.

These features make MATLAB a powerful platform for engineers, researchers, and students to analyse data, create models, and develop scientific applications.

1.2 Advantages and Disadvantages of MATLAB:

MATLAB is a powerful software widely used for numerical computation, data analysis, and simulation. Like any tool, it has both advantages and limitations.

Advantages of MATLAB:

1. **Easy to Learn and Use:**
   MATLAB has a simple, high-level syntax which makes programming and mathematical operations easy for beginners.

2. **Excellent Visualization:**
   It provides high-quality 2D and 3D plotting tools for graphs, charts, and simulation results.

3. **Rich Toolboxes:**
   MATLAB offers specialized toolboxes for areas like signal processing, image processing, control systems, and machine learning.

4. **Strong for Numerical Computing:**
   It is highly efficient for matrix operations, mathematical calculations, and algorithm development.

5. **Platform Independent:**
   MATLAB runs on Windows, macOS, and Linux, making it portable and flexible.

Disadvantages of MATLAB:

1. **Costly Software:**
   MATLAB requires expensive licenses, which may not be affordable for all students or small organizations.

2. **Slower than Low-Level Languages:**
   Compared to C, C++, or Java, MATLAB can be slower for certain computational tasks.

3. **Limited for General-Purpose Programming:**
   MATLAB is mainly designed for numeric and scientific computing, not for creating full scale applications.

4. **Requires More Memory:**
   MATLAB programs often consume more RAM than programs written in lower-level languages.

5. **Proprietary Software:**
   Since it is not open-source, users cannot modify the core functionalities or access the internal source code.

1.3 MATLAB Environment:

The **MATLAB Environment** is an integrated workspace that provides all the tools needed for writing programs, performing calculations, visualizing data, and managing files. It offers an easy-to-use interface that helps users interact with MATLAB functions efficiently. The main components of the MATLAB environment are:

1. **Command Window:**
   It is the main area where users type commands and execute them immediately. It is used

for quick calculations and running functions.

2. **Workspace:**
   Displays all the variables currently in memory. Users can view, modify, or clear variables during program execution.

3. **Command History:**
   Stores all previously executed commands. Users can reuse past commands by selecting them from the history.

4. **Editor/Script Window:**
   This is where users write, edit, and save programs (scripts and functions). It supports debugging tools like breakpoints and error detection.

5. **Current Folder / File Browser:**
   Shows the files and folders in the working directory. It helps users manage program files, data files, and project folders.

6. **Figure Window:**
   Displays graphs, plots, and visual outputs generated by MATLAB commands.

These components make the MATLAB environment interactive, user-friendly, and powerful for performing numerical analysis, simulations, and application development.

1.4 Using MATLAB Scratch Pad:

The **MATLAB Scratch Pad** refers to the use of the **Command Window** as a temporary workspace where users can quickly test commands, perform calculations, and experiment with small pieces of code without creating a formal script or file. It acts like a rough notebook where programmers can try ideas before writing the final program in the Editor.

Key Points About Using MATLAB Scratch Pad:

1. **Quick Testing of Commands:**
   Users can type any MATLAB command directly into the Command Window to check results instantly. This is useful for verifying syntax, testing functions, and performing small calculations.

2. **Temporary Workspace:**
   The variables created in the Scratch Pad exist only during the session. They can be viewed in the Workspace and are automatically removed when MATLAB is closed unless saved intentionally.

3. **Ideal for Experimentation:**
   It allows users to experiment with mathematical expressions, matrix operations, and plotting commands without worrying about file creation or program structure.

4. **Immediate Output:**
   Results appear instantly in the Command Window, helping beginners understand how MATLAB processes commands and giving quick feedback during problem-solving.

5. **Helps in Script Development:**
   Before writing a full script in the Editor, users can try small code snippets in the Scratch Pad to ensure correctness, which reduces debugging time and improves program accuracy.

Using the MATLAB Scratch Pad makes learning and working with MATLAB faster, easier, and more interactive.

1.5 Variables and Arrays:

Variables:

A **variable** is a reserved memory location used to store data during program execution. It acts as a container that holds values, which can be changed or updated as the program runs. Variables allow programmers to store input data, intermediate results, and final outputs, making programs dynamic and flexible.

Key Points about Variables:

1. **Definition and Purpose:**
   A variable is a named reference to a value stored in memory. It helps in storing numbers, text, or other data types.

2. **Naming Rules:**
   Variable names must start with a letter, can include digits and underscores, and should not use reserved keywords of the programming language.

3. **Dynamic Typing (in languages like Python/MATLAB):**
   The type of a variable is decided at runtime. A variable can store different types of values at different times.

4. **Assignment of Values:**
   Values are assigned using the assignment operator (e.g., `=`). Example: `x = 10`.

5. **Usage in Programs:**
   Variables are used for calculations, storing user input, controlling loops, and passing data within functions.

Variables are fundamental building blocks that make programs interactive, reusable, and easier to understand.

Arrays:

An **array** is a collection of multiple values stored in a single variable. All the elements in an array are usually of the same data type and are stored in continuous memory locations. Arrays help in efficient storage, access, and manipulation of large sets of data such as lists of numbers, matrices, or sequences.

Key Points about Arrays:

1. **Definition:**
   An array holds multiple data elements under one name, accessed using an index (e.g., `A(1)`, `A[0]`).

2. **Types of Arrays:**
   They can be one-dimensional (lists), two-dimensional (matrices), or multi-dimensional for complex data.

3. **Efficient Data Handling:**
   Arrays allow fast processing of large quantities of data, especially useful in mathematical and scientific computations.

4. **Indexing:**
   Elements are accessed based on their position. Some languages use 1-based indexing (MATLAB), while others use 0-based indexing (Python).

5. **Operations on Arrays:**
   Common operations include insertion, deletion, traversal, searching, sorting, and mathematical operations (like matrix multiplication in MATLAB).

Arrays simplify data organization and are essential for numerical analysis, data science, simulations, and algorithm development.

1.6 Multidimensional Arrays:

A **multidimensional array** is an array that contains more than one level or dimension of data. Unlike one-dimensional arrays that store elements in a single row, multidimensional arrays organize data in multiple directions, such as rows and columns (2D), or in layers (3D). They are widely used in scientific computing, image processing, simulations, and mathematical modeling.

Key Points about Multidimensional Arrays:

1. **Definition:**
   A multidimensional array stores data in two or more dimensions. Common forms include 2D arrays (matrices) and 3D arrays (cubes of data).

2. **Two-Dimensional Arrays (Matrix):**
   A 2D array organizes data in rows and columns. It is useful for representing tables, grids, or mathematical matrices.

3. **Three-Dimensional Arrays:**
   A 3D array adds another dimension called depth or layers. It is used in applications like image stacks, scientific simulations, and volume data.

4. **Indexing:**
   Elements are accessed using multiple indices.

   MATLAB example: `A(2, 3)` accesses row 2, column 3.
   For 3D arrays: `A(2, 3, 1)` accesses an element from the first layer.

5. **Applications:**
   Multidimensional arrays are essential in engineering, graphics, machine learning, signal processing, and data analysis where large datasets require structured storage.

Multidimensional arrays help represent and process complex data efficiently, making them an important concept in numerical and scientific programming.

1.7 Scalar and Array Operations:

Scalar Operations:

Scalar operations are operations performed on **single numerical values** (called scalars). A scalar represents one value such as an integer, float, or real number. These operations include basic arithmetic, relational, and logical operations applied to individual values.

Key Points about Scalar Operations:

1. **Definition:**
   Scalar operations involve computations on single values, such as `a = 5`, `b = 3`, and operations like `a + b`.

2. **Arithmetic Operations:**
   Scalars support operations like addition (`+`), subtraction (`−`), multiplication (`*`), division (`/`), and exponentiation (`^`).
   Example: `x = 10 + 5` gives `15`.

3. **Relational Operations:**
   Scalars are compared using operators like `<`, `>`, `==`, `<=`, `>=`.
   Example: `5 > 3` results in `true`.

4. **Logical Operations:**
   Logical operators such as `AND`, `OR`, and `NOT` can be applied to scalar Boolean values.
   Example: `true && false`.

5. **Use in Expressions:**
   Scalar operations are used in formulas, control expressions, mathematical calculations, and intermediate steps in programs.

Scalar operations form the foundation for computation and are essential for performing mathematical and decision-making tasks in programming.

Array Operations:

Array operations are operations performed on **multiple elements stored in an array**. These operations can be element-wise or matrix-based, depending on the programming environment (like MATLAB, Python NumPy, etc.).

Key Points about Array Operations:

1. **Definition:**
   Array operations apply arithmetic or logical operations to entire arrays containing multiple elements.

2. **Element-Wise Operations:**
   Operations are applied to each element independently.
   Example in MATLAB: `A .* B` multiplies each element of array `A` with the corresponding element of array `B`.

3. **Matrix Operations:**
   Arrays can represent matrices, allowing matrix multiplication (`A * B`), transposition (`A'`), and inverse (`inv(A)`).

4. **Broadcasting (Automatic Size Matching):**
   Some languages (like Python NumPy) allow operations between arrays of different shapes by automatically expanding dimensions.

5. **Applications:**
   Array operations are widely used in data analysis, image processing, scientific computation, simulations, and machine learning where large datasets must be processed efficiently.

Array operations make programming faster and more powerful by allowing entire datasets to be manipulated using a single command.

Unit II

2. Introduction to Digital Image Representation

2.1 Elements of Digital Image Processing System
2.2 Digital Image Representation
2.3 Reading, displaying and writing images
2.4 Data classes and Image types
2.5 Converting between data classes and image types
2.6 Introduction to M-function Programming

2.1 Elements of Digital Image Processing System:

A **Digital Image Processing (DIP) System** consists of several components that work together to capture, process, analyze, and display digital images. These elements help perform tasks such as enhancement, restoration, compression, and interpretation of images.

Key Elements of a Digital Image Processing System:

1. **Image Acquisition:**
   This is the first step where an image is captured using devices like cameras, scanners, or sensors. It includes converting optical images into digital form using sampling and quantization.

2. **Image Storage:**
   After acquisition, digital images are stored in memory devices such as RAM, hard disks, cloud storage, or specialized image databases. Efficient storage techniques may include compression.

3. **Image Processing Hardware:**
   This includes the processors (like GPUs, DSPs, microprocessors) used to perform operations such as filtering, enhancement, resizing, and transformations. Hardware accelerates processing speed.

4. **Image Processing Software:**
   Software provides tools and algorithms for tasks like image enhancement, segmentation, edge detection, noise removal, and pattern recognition. Examples include MATLAB, OpenCV, and Python libraries.

5. **Display and Visualization:**
   Images are displayed on monitors, screens, or printers for interpretation. High-resolution displays allow users to analyze fine details and processed results.

6. **Image Analysis and Interpretation:**
   In this stage, meaningful information is extracted from the image. It includes object detection, measurement, classification, and decision-making (used in medical imaging, biometrics, remote sensing, etc.).

These elements together form a complete digital image processing system that captures, processes, analyzes, and displays images for various scientific, industrial, and medical applications.

2.2 Digital Image Representation:

Digital Image Representation refers to the method of storing, displaying, and processing images in a computer system. A digital image is represented as a two-dimensional array of small picture elements called

pixels, where each pixel holds a numerical value that represents the intensity or color at that point.

Key Points about Digital Image Representation:

1. **Pixel (Picture Element):**
   A digital image is made up of tiny dots called pixels. Each pixel contains an intensity value (for grayscale) or color value (for color images). Higher pixel count gives better image quality.

2. **Resolution:**
   It refers to the number of rows and columns of pixels in an image (e.g., 1920×1080). Higher resolution means more pixels and clearer, more detailed images.

3. **Intensity Levels / Gray Levels:**
   Grayscale images represent intensity values using a range of gray shades. Most digital images use 8-bit representation, giving **256 gray levels** (0 = black, 255 = white).

4. **Color Image Representation:**
   Color images are usually represented using the **RGB model**, where each pixel has three components:
   R (Red)
   G (Green)
   B (Blue)
   Each component typically uses 8 bits, making a total of **24 bits per pixel**.

5. **Bit Depth:**
   Bit depth defines how many bits are used to represent each pixel. Higher bit depth allows more color or intensity variations.
   Examples:
   1-bit = 2 levels (black/white)
   8-bit = 256 gray levels
   24-bit = 16 million colors

Digital Image Representation allows computers to store, display, and manipulate images efficiently for various applications like medical imaging, remote sensing, photography, and machine vision.

2.3 Reading, displaying and writing images:

1. Reading Images:

Reading an image means importing a digital image from a file into the computer's memory so that it can be processed using image processing techniques. The process converts the file format (such as JPG, PNG, BMP) into a numerical matrix of pixel values.

Key Points:

1. **Definition:**
   Reading an image refers to loading an image file and converting it into a matrix where each element represents a pixel.

2. **Function Used (in MATLAB/Python):**
   MATLAB: `imread('image.jpg')`
   Python OpenCV: `cv2.imread('image.jpg')`

3. **Matrix Representation:**
   A grayscale image becomes a 2D matrix of intensity values.
   A color image becomes a 3D matrix (rows × columns × 3 channels).

4. **Importance:**
   Reading is the first step in any digital image processing task such as enhancement, filtering, or segmentation.

5. **Supports Multiple Formats:**
   The system can read several formats like JPG, PNG, BMP, TIFF, etc., making it flexible for various applications.

2. Displaying Images:

Displaying an image means showing the image on a screen or window so that the user can view it. This helps in visualizing the results of processing techniques like enhancement, noise removal, or filtering.

Key Points:

1. **Definition:**
   Displaying an image is the process of presenting a digital image visually on a monitor or GUI window.

2. **Function Used:**
   MATLAB: `imshow(image)`
   Python OpenCV: `cv2.imshow("Title", image)`

3. **Visualization of Pixel Data:**
   The pixel matrix is converted into a visual form where intensity or color values become viewable images.

4. **Used for Analysis:**
   Displaying is essential for checking image quality, verifying results, and comparing before-and-after processing outputs.

5. **Multiple Display Options:**
   Systems support zooming, adjusting colormaps, showing histograms, and displaying multiple images in subplots.

3. Writing Images:

Writing an image refers to saving the processed image back to a file. It stores the modified pixel matrix as an image in a chosen format.

Key Points:

1. **Definition:**
   Writing an image means exporting the pixel matrix to an external file, such as JPG, PNG, BMP, or TIFF.

2. **Function Used:**
   MATLAB: `imwrite(image, 'output.jpg')`
   Python OpenCV: `cv2.imwrite('output.jpg', image)`

3. **Preserves Processing Results:**
   After performing enhancement, filtering, segmentation, etc., the final result is saved for future use.

4. **Supports Compression:**
   Writing allows saving images in different formats that may use compression (like JPG) or lossless storage (like PNG/TIFF).

5. **Applications:**
   Writing is used in medical imaging, document scanning, graphics design, and machine learning datasets where processed images must be stored.

2.4 Data classes and Image types:

1. Data Classes:

In Digital Image Processing, **data classes** refer to the types of numerical data used to store pixel values in an image. Different data classes determine the range, precision, and memory required for representing image pixels.

Key Points about Data Classes:

1. **Definition:**
   Data classes specify the format and size of pixel values stored in an image, such as integer or floating-point types.

2. **Common Data Classes:**

   uint8:
   8-bit unsigned integer (0–255). Most commonly used for grayscale and RGB images.

   uint16:
   16-bit unsigned integer (0–65,535). Used for high-precision images like medical and scientific data.

   int16:
   16-bit signed integer (−32,768 to 32,767). Used in specialized applications.

   single (float32) / double (float64):
   Floating-point classes used for calculations, filtering, normalization, or scientific processing.

3. **Memory Requirement:**
   Higher data classes (like 16-bit and 32-bit) require more memory but provide higher accuracy and dynamic range.

4. **Purpose:**
   Data classes help ensure correct processing results because many algorithms depend on pixel value range and precision.

5. **Usage:**
   Different data classes are used in medical imaging, satellite images, scientific imaging, and applications requiring high dynamic range.

Data classes are essential for selecting the correct representation of pixel values based on required accuracy and processing needs.

2. Image Types:

Image types describe the **structure, format, and characteristics** of images used in digital image processing. Different image types store and represent image data in different ways.

Key Types of Digital Images:

1. **Binary Image:**
   Each pixel has only two values: 0 and 1 (black or white). Used in document processing, thresholding, and shape analysis.

2. **Grayscale Image:**
   Pixels represent shades of gray with intensity values typically ranging from 0 to 255 (8 bit). Commonly used in medical and industrial imaging.

3. **RGB Color Image:**
   Each pixel contains three components: Red, Green, and Blue. Represented as a 3D array (rows × columns × 3). Used in natural photographs.

4. **Indexed Image:**
   Pixels store color indices instead of actual RGB values. A colormap (lookup table) maps each index to an RGB color. Used for reducing memory usage.

5. **Multispectral / Hyperspectral Image:**
   Contains multiple bands beyond visible colors, captured by sensors. Used in remote sensing, agriculture, and scientific imaging.

Importance:

Different image types are used depending on storage needs, processing requirements, and application domains like medical imaging, biometrics, satellite imaging, and computer vision.

2.5 Converting between data classes and image types:

1. Converting Between Data Classes:

In Digital Image Processing, images may need to be converted from one **data class** to another depending on the requirements of algorithms, memory usage, or precision. Converting data classes changes the pixel value range and storage format without altering the actual visual content of the image.

Key Points:

1. **Definition:**
   Conversion between data classes refers to changing the numerical representation of pixel values from one format (e.g., uint8) to another (e.g., double or uint16).

2. **Common Conversions:**
   `uint8` ↔ `double`
   `uint8` ↔ `uint16`
   `double` ↔ `single`
   These conversions help match image data with specific processing functions.

3. **Function Used:**
   In MATLAB: `im2double()`, `im2uint8()`, `im2uint16()`, `im2single()`

4. **Purpose:**
   Algorithm compatibility: Many algorithms require images in double precision.
   Memory management: Converting to `uint8` reduces file size.
   Precision control: Scientific or medical imaging uses `uint16` for high dynamic range.

5. **Range Adjustment:**
   When converting, pixel values may be scaled:
   `uint8 (0–255)` → `double (0–1)`

`uint16 (0–65535)` → `uint8 (0–255)`
Proper scaling ensures accurate brightness and contrast.

Converting between data classes allows efficient processing, accurate results, and compatibility with different algorithms.

2. Converting Between Image Types:

Digital images can exist in different **image types** such as binary, grayscale, RGB, and indexed images. Converting between image types helps in applying appropriate processing operations or visualizing images in different forms.

Key Points:

1. **Definition:**
   Conversion between image types means changing the structure and representation of an image, such as converting an RGB image to grayscale or a grayscale image to a binary image.

2. **Common Conversions:**
   RGB → Grayscale: Converts 3-channel color images to single-channel intensity images.
   Grayscale → Binary: Uses thresholding to convert intensity values to 0 or 1.
   Indexed → RGB: Applies a colormap to convert index values into actual RGB colors.
   RGB → Indexed: Reduces the number of colors for memory-efficient storage.

3. **Functions Used (MATLAB):**
   `rgb2gray()`
   `im2bw()` or `imbinarize()`
   `ind2rgb()`
   `rgb2ind()`

4. **Purpose:**
   Simplify processing tasks (e.g., edge detection works better on grayscale than RGB).
   Improve storage efficiency (indexed images use less memory).
   Prepare images for segmentation, enhancement, or analysis.

5. **Applications:**
   Used in computer vision, medical imaging, OCR, machine learning, compression, and graphics where images must be converted to suitable formats before processing.

Conversion between image types ensures that images match the required format for specific algorithms and processing tasks.

2.6 Introduction to M-function Programming:

M-function programming refers to writing user-defined functions in MATLAB using files that have the extension **".m"**. These files contain a set of commands grouped together to perform a specific task. M functions help modularize code, improve reusability, and make programs easier to understand and maintain.

Key Points:

1. **Definition:**
   An M-function is a MATLAB program written in a separate `.m` file that starts with the keyword `function`. It accepts input arguments, processes them, and returns output values.

2. **Structure of an M-Function:**

Every M-function includes:

A **function definition line**

Input arguments

Output arguments

A **body** containing MATLAB statements

Example:

`function y = square(x)`

3. **Separation from Script Files:**

Unlike script files that run sequential commands, M-functions are reusable blocks of code that can be called repeatedly with different inputs from the Command Window or other programs.

4. **Advantages:**

Promotes modular programming

Reduces code repetition

Easy to debug and maintain

Allows building complex applications using smaller functions

5. **Types of M-Files:**

MATLAB supports:

Function M-files (user-defined functions)

Script M-files (sequences of commands)

M-functions are essential for creating custom operations and advanced programs.

M-function programming enables structured, efficient, and reusable coding in MATLAB, making it an important feature for scientific and engineering applications.

Unit III

3. Intensity Transformation and Spatial Filtering

3.1 Background : Intensity Transformation Functions
3.2 Using imadjust( ), Using log( )
3.3 Histogram Processing and function plotting, Spatial filtering
3.4 Linear spatial filtering, Non-Linear spatial filtering

3.1 Background : Intensity Transformation Functions

Intensity Transformation Functions are basic operations in Digital Image Processing used to modify the pixel intensity values of an image. These functions apply a mathematical transformation to each pixel to improve image appearance or prepare it for further processing. They operate directly on the intensity values without changing the pixel positions.

Key Points About Intensity Transformation Functions:

1. **Definition:**
   Intensity transformation functions map input pixel values (**r**) to new output values (**s**) using a transformation function **s = T(r)**. Each pixel is modified independently.

2. **Purpose:**
   They are used to enhance contrast, brighten or darken images, correct lighting issues, highlight certain details, and prepare images for segmentation or analysis.

3. **Common Types of Transformations:**
   Linear Transformations: Brightness adjustment, contrast stretching.
   Logarithmic Transformations: Enhance dark regions and compress bright regions.
   Power-Law (Gamma) Transformations: Control overall image brightness using gamma correction.
   Piecewise Linear Transformations: Includes thresholding and gray-level slicing.

4. **Point Processing:**
   Intensity transformations are point-processing operations, meaning each output pixel depends only on the corresponding input pixel, making them simple and fast.

5. **Applications:**
   Used in medical imaging, satellite images, photography, and preprocessing steps before edge detection, segmentation, and pattern recognition.

Intensity transformation functions play a fundamental role in enhancing image quality and improving visibility of important details in digital images.

3.2 Using imadjust( ), Using log( ):

1. Using `imadjust()`:

`imadjust()` is a MATLAB function used to enhance the contrast of an image by adjusting the intensity values. It performs **intensity mapping** by stretching or compressing the range of pixel values.

Key Points:

1. **Definition:**
   `imadjust()` maps the intensity values of an image to a new range, improving brightness and contrast. The transformation is of the form:
   s = T(r), where `r` is input intensity and `s` is output intensity.

2. **Syntax:**
   `J = imadjust(I, [low_in high_in], [low_out high_out])`

3. **Functionality:**
   Pixels below `low_in` are mapped to `low_out`.
   Pixels above `high_in` are mapped to `high_out`.
   Pixels between them are linearly stretched.

4. **Uses:**
   Brightening dark images
   Increasing contrast of low-contrast images
   Normalizing uneven lighting
   Adjusting specific intensity ranges

5. **Example:**
   `J = imadjust(I, [0.2 0.8], [0 1]);`
   This stretches the contrast by expanding the middle intensities into the full 0–1 range.

`imadjust()` is widely used for preprocessing before segmentation, enhancement, and edge detection.

2. Using `log()` (Logarithmic Transformation):

Logarithmic transformation enhances the details in **dark regions** of an image while compressing the brightness of **high-intensity pixels**. It is useful for images with **large dynamic ranges** (very bright and very dark areas in the same image).

Key Points:

1. **Definition:**
   A log transformation applies the formula:
   $$s = c \cdot \log(1 + r)$$
   where `r` is the pixel intensity and `c` is a scaling constant.

2. **Purpose:**
   Expands low-intensity values (makes dark areas brighter)
   Compresses high-intensity values (reduces brightness of very bright areas)
   Enhances details in shadows

3. **MATLAB Usage:**
   ```
   I = im2double(I);
   J = c * log(1 + I);
   ```

   `c` is chosen to normalize the output.

4. **Applications:**
   Medical imaging (X-rays, CT scans)
   Satellite images (night-time, low illumination)
   Images with high contrast variations
   Detecting details hidden in dark regions

5. **Characteristics:**
   Non-linear transformation
   Useful for compressing large dynamic ranges
   Often applied with normalization for best visual results

Log transformation is a powerful tool for enhancing low-intensity regions and is a standard technique in image enhancement.

3.3 Histogram Processing and function plotting, Spatial filtering:

1. Histogram Processing:

Histogram processing is a fundamental technique in digital image processing used to analyze and enhance the contrast of an image based on the distribution of its pixel intensities.

Definition:

A histogram is a graphical representation of the frequency of intensity values in an image. It shows how many pixels correspond to each gray level.

Key Concepts:

1. **Histogram Equalization:**
   Enhances contrast by spreading out frequently occurring intensity values. It produces an image with uniform histogram distribution.

2. **Histogram Matching (Specification):**
   Adjusts the image so that its histogram matches a desired target histogram.

3. **Histogram Stretching (Normalization):**
   Expands the range of intensity values to cover the full dynamic range (e.g., 0–255), improving contrast.

4. **Cumulative Distribution Function (CDF):**
   Used in calculating equalized intensity values.

Uses:

Improving visibility of details in low-contrast images
Medical imaging, satellite imaging, pattern recognition
Preprocessing for further image analysis

Conclusion:

Histogram processing is widely used to enhance image contrast and reveal important features that may not be visible in the original image.

2. Function Plotting:

Function plotting in MATLAB is used to visualize mathematical expressions or relationships between variables.

Key Points:

1. **Definition:**
   It refers to graphically representing a function (y=f(x)) using MATLAB commands.

2. **Basic Plot Command:**

```
x = 0:0.1:10;
y = sin(x);
plot(x,y)
```

   This generates a 2D plot of the sine function.

3. **Labels and Titles:**
   Commands like `xlabel`, `ylabel`, `title`, and `grid` improve readability.

4. **Multiple Plots:**
MATLAB allows plotting several functions on the same graph using `hold on` or by specifying multiple parameters in `plot()`.

5. **Applications:**
Used in mathematics, engineering analysis, signal processing, and algorithm visualization.

Conclusion:
Function plotting helps understand trends, compare functions, and visually interpret mathematical results.

3. Spatial Filtering:
Spatial filtering is an image processing operation that modifies pixel intensity values based on the values of neighboring pixels.

Definition:
A filter (also called a mask or kernel) is applied to an image by sliding it across the image and computing a new value for each pixel.

Types of Spatial Filters:
1. **Linear Filters (Convolution-based):**
Smoothing/Blur Filters: Reduce noise and details using averaging or Gaussian filters.
Sharpening Filters: Highlight edges using Laplacian or high-pass filters.

2. **Nonlinear Filters:**
Median Filter: Removes salt-and-pepper noise by taking the median of neighbourhood pixels.
Max/Min Filters: Used in morphological operations.

Operations Used:
Convolution:
Applying the filter weights to corresponding image pixels.

Correlation:
Similar to convolution but without kernel flipping.

Applications:
Noise reduction
Edge detection
Image smoothing and sharpening
Feature enhancement

Conclusion:
Spatial filtering improves image quality and extracts important features, making it essential in many image analysis tasks.

3.4 Linear spatial filtering, Non-Linear spatial filtering:

1. Linear Spatial Filtering:
Linear spatial filtering is an image processing technique in which the output pixel value is a **linear combination of the neighboring pixel intensities**. It uses a filter (kernel or mask) that is applied over the image through **convolution** or **correlation**.

Key Points:

1. **Definition:**

   A linear filter replaces each pixel with a weighted sum of its neighborhood. The weights are defined by a filter mask.

2. **Convolution Operation:**

   The mask moves over the image, multiplies its values with the underlying pixels, and adds the results to produce a new pixel value.

   Example:

$$g(x,y) = \sum i \sum j \;\; w(i,j) f(x-i, y-j)$$

3. **Types of Linear Filters:**

   Smoothing (Low-pass) Filters: Reduce noise and blur the image.

   Examples: Averaging filter, Gaussian filter.

   Sharpening (High-pass) Filters: Enhance edges and fine details.

   Examples: Laplacian filter, Sobel and Prewitt operators.

4. **Applications:**

   Noise reduction
   Edge detection
   Image enhancement
   Smoothing or blurring

Conclusion:

Linear spatial filtering is widely used because it is mathematically simple, easy to implement, and effective for tasks like smoothing and sharpening images.

2. Non-Linear Spatial Filtering:

Non-linear spatial filtering processes an image using nonlinear operations, meaning the output pixel is **not** a linear combination of neighbourhood pixels. These filters are especially useful for noise removal while preserving edges.

Key Points:

1. **Definition:**

   A non-linear filter replaces each pixel with a value derived from its neighborhood using a **nonlinear** mathematical function (median, max, min, etc.).

2. **Common Non-linear Filters:**

   Median Filter: Replaces pixel with median of neighborhood. Very effective for salt and-pepper noise.

   Max Filter: Selects highest intensity in the window (used in dilation).

   Min Filter: Selects lowest intensity in the window (used in erosion).

   Adaptive Filters: Adjust based on local image statistics.

3. **Advantages:**

   Preserves edges better than linear smoothing filters
   Removes impulsive noise effectively

4. **Applications:**

   Removing salt-and-pepper noise
   Edge-preserving smoothing
   Morphological image processing
   Preprocessing for segmentation

Conclusion:

Non-linear spatial filtering is powerful for noise removal and edge preservation, making it essential in applications where linear filters fail to maintain important image details.

Unit IV

4. Frequency Domain Processing

4.1 Introduction to Discrete Fourier Transformation (DFT)
4.2 Computing and visualizing 1D-DFT
4.3 Computing and visualizing 2D-DFT
4.4 Filtering in frequency domain
4.5 Color Image Representation
4.6 Converting to other color spaces
4.7 The Basics of color image processing

4.1 Introduction to Discrete Fourier Transformation (DFT):

The **Discrete Fourier Transformation (DFT)** is a fundamental mathematical technique used in digital signal and image processing to convert a signal from the **spatial (time or image) domain** into the **frequency domain**. It analyzes how different frequency components contribute to the overall signal.

Definition:

DFT transforms a sequence of discrete values into an equal-length sequence representing its frequency components. For an input sequence $f(n)$, the DFT is given by:

$$F(k) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi kn/N}$$

Here,

$f(n)$: Input signal
$F(k)$: Frequency-domain output
N: Number of samples

Key Concepts:

1. **Frequency Representation:**
   DFT shows how much of each frequency is present in the signal, separating it into low, medium, and high frequencies.

2. **Reversibility:**
   The original signal can be reconstructed using the inverse DFT (IDFT).

3. **Magnitude and Phase:**
   Each frequency component has a **magnitude** (strength) and **phase** (position shift).

4. **Complex Output:**
   The DFT produces complex numbers due to sinusoidal decomposition using sine and cosine waves.

Applications:

Image filtering and enhancement
Noise reduction
Feature extraction
Compression
Pattern and frequency analysis

Conclusion:

DFT is a powerful tool that converts signals into their frequency components, enabling efficient analysis, filtering, and processing of digital images and signals.

4.2 Computing and visualizing 1D-DFT:

1. Computing 1D-DFT:

The **1-Dimensional Discrete Fourier Transform (1D-DFT)** is used to convert a discrete signal from the **time (or spatial) domain** into the **frequency domain**. Computing the DFT involves applying a mathematical formula that breaks the signal into sinusoidal components of different frequencies.

DFT Formula:

For a discrete sequence $f(n)$ of length N, the DFT is computed as:

$$F(k) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi kn/N}, \quad k = 0, 1, 2, \ \dots, \ N-1$$

Steps in Computing 1D-DFT:

1. **Take the input signal** (e.g., a list of intensity or amplitude values).
2. **Multiply each sample** with complex exponential basis functions.
3. **Sum all the products** to obtain a frequency component $F(k)$.
4. **Repeat for all values of k** to compute the full spectrum.
5. The output is a set of **complex values** representing magnitude and phase.

Characteristics:

Produces both **real and imaginary parts**.
Shows how strongly each frequency exists in the signal.
Time-domain convolutions become multiplications in frequency domain.

MATLAB Example:

```
x = [1 2 3 4];
X = fft(x);
```

Conclusion:

Computing 1D-DFT is essential for analyzing the frequency content of signals, enabling filtering, compression, and spectral analysis.

2. Visualizing 1D-DFT:

Once the 1D-DFT is computed, it is important to **visualize** the frequency components for better understanding and analysis. Visualization shows the **magnitude** and **phase** of different frequency elements present in a signal.

Key Concepts in Visualization:

1. **Magnitude Spectrum:**

   Displays the strength (amplitude) of each frequency component.
   Magnitude is calculated as:

$$|F(k)| = \sqrt{(real\ (F(k)))^2 + (imag\ (F(k)))^2}$$

2. **Phase Spectrum:**

   Shows the phase shift of each frequency component.
   Phase is calculated as:

$$< F(k) = tan^{-1}\left(\frac{imag\ (F(k))}{real\ (F(k))}\right)$$

3. **Frequency Axis:**

   The x-axis represents discrete frequency values from 0 to $N-1$.

MATLAB Example for Visualization:

```
x = [1 2 3 4];
X = fft(x);
```

```
subplot(2,1,1);
plot(abs(X));   % Magnitude spectrum
title('Magnitude Spectrum');

subplot(2,1,2);
plot(angle(X)); % Phase spectrum
title('Phase Spectrum');
```

Importance of Visualization:

Helps identify dominant frequencies
Useful for detecting noise components
Allows comparison of signals
Essential in communications, audio processing, and image analysis

Conclusion:

Visualizing 1D-DFT provides clear insight into both the magnitude and phase characteristics of a signal, making it a crucial step in frequency-domain analysis.

4.3 Computing and visualizing 2D-DFT:

The **2-Dimensional Discrete Fourier Transform (2D-DFT)** is used in digital image processing to convert an image from the **spatial domain** (pixels) to the **frequency domain**. It helps analyze how different frequency components contribute to the image structure.

1. Computing 2D-DFT:

For an image $f(x, y)$ of size $M \times N$, the 2D-DFT is computed using:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

Steps:

1. Take the input image as a 2D array of pixel values.
2. Multiply each pixel by complex exponential basis functions.
3. Sum the products over all rows and columns to obtain frequency components.
4. Output is a complex-valued matrix representing frequency content.
5. In MATLAB, 2D-DFT is computed using `fft2()`.

MATLAB Example:

```
img = imread('image.jpg');
F = fft2(img);          % compute 2D-DFT
F_shift = fftshift(F);  % shift low frequencies to center
```

2. Visualizing 2D-DFT:

Visualization helps interpret frequency information such as edges, textures, and smooth areas.

Key Concepts:

1. **Magnitude Spectrum:**

   Shows the strength of each frequency component. High values correspond to strong edges or details.

$$|F(u, v)| = \sqrt{(Real)^2 + (Imag)^2}$$

2. **Log Transform:**

   Used because magnitude values vary widely.

$$\log(1 + |F(u, v)|)$$

3. **Phase Spectrum:**
Represents spatial alignment; essential for reconstructing the image.

MATLAB Example for Visualization:

```
figure;
imshow(log(1 + abs(F_shift)), []);
title('Magnitude Spectrum');

figure;
imshow(angle(F_shift), []);
title('Phase Spectrum');
```

Interpretation:
Bright areas in magnitude spectrum = strong frequency components.
Center represents low frequencies (smooth areas).
Corners represent high frequencies (edges, sharp transitions).

Conclusion:
Computing and visualizing 2D-DFT provides a powerful way to analyze image frequency content, identify edges and textures, and perform operations such as filtering, enhancement, and compression.

4.4 Filtering in frequency domain:

Filtering in the frequency domain is a technique used in digital image processing to **modify specific frequency components** of an image to achieve effects such as smoothing, sharpening, and noise reduction.

Concept:
An image is first transformed from the spatial domain to the **frequency domain** using the **Discrete Fourier Transform (DFT)**. In this domain, image details are represented as frequency components:
Low frequencies: smooth areas
High frequencies: edges, noise, fine details

Filtering is done by **multiplying** the image's Fourier transform with a **frequency-domain filter mask**.

Steps in Frequency-Domain Filtering:

1. **Take the image** $f(x, y)$.

2. **Compute its 2D-DFT:**
$$F(u, v) = DFT\{f(x, y)\}$$

3. **Design a filter** $H(u, v)$ (low-pass, high-pass, band-pass, etc.).

4. **Multiply:**
$$G(u, v) = H(u, v) \cdot F(u, v)$$

5. **Apply inverse DFT** to obtain the filtered image:
$$g(x, y) = IDFT\{G(u, v)\}$$

Types of Frequency-Domain Filters:

1. **Low-Pass Filters (LPF):**
   Allow low frequencies; remove noise and smooth the image.
   Examples: Ideal LPF, Gaussian LPF.

2. **High-Pass Filters (HPF):**
   Allow high frequencies; enhance edges and fine details.
   Examples: Ideal HPF, Butterworth HPF.

3. **Band-Pass and Band-Stop Filters:**
   Allow or block specific frequency ranges.

Advantages:

More precise control over frequency components
Effective for smoothing and sharpening
Useful in image enhancement and restoration

Conclusion:

Filtering in the frequency domain is a powerful method that uses the Fourier transform to selectively modify frequency components, enabling tasks like noise removal, edge enhancement, and overall image quality improvement.

4.5 Color Image Representation:

Color image representation refers to the method by which color information is stored, processed, and displayed in digital images. Unlike grayscale images that contain only intensity values, color images consist of multiple components representing different color channels.

1. RGB Color Model:

The most widely used representation is the **RGB (Red, Green, Blue)** model.

A color image is stored as **three separate 2D matrices** (one for each color channel).
Each pixel is represented by a combination of **R, G, and B intensities**, typically in the range 0 - 255.
These three channels combine to produce a full-color image.

2. True Color Image:

A standard color image uses **24-bit representation**, where:

8 bits → Red
8 bits → Green
8 bits → Blue
This allows **16.7 million** possible colors (2^{24}).

3. Other Color Models:

Besides RGB, several models are used for specific applications:

HSV (Hue, Saturation, Value): represents color in terms of shade, colorfulness, and brightness.
CMY/CMYK: used in printing.
YCbCr: used in video compression and broadcasting.
Different models separate brightness information from color information, useful for enhancement and compression.

4. Pixel Structure:

Each pixel in a color image is represented as a **vector**:

$$Pixel = [R, G, B]$$

Thus, a color image can be seen as a **3D array** (height × width × 3).

5. Applications:

Computer vision
Image editing
Medical imaging
Remote sensing
Graphics and multimedia

Conclusion:

Color image representation uses multiple color channels (typically RGB) to store and display color information. This multidimensional data structure allows accurate reproduction, processing, and analysis of colored images.

4.6 Converting to other color spaces:

Color images are most commonly represented in the **RGB color model**, but many image processing applications require converting images into other color spaces such as **HSV, YCbCr, CMY, or Lab**. This conversion allows better manipulation of brightness, contrast, and color information.

1. Need for Color Space Conversion:

Different color spaces separate color and intensity information differently. Converting RGB images to other color models helps in:

Image segmentation
Object recognition
Compression
Color enhancement
Video processing

2. Common Color Spaces and Their Use:

1. **HSV (Hue, Saturation, Value):**
   Separates color information (Hue) from brightness (Value).
   Useful in color-based segmentation, tracking, and filtering.

2. **YCbCr:**
   Separates luminance (Y) from chrominance (Cb, Cr).
   Used in image and video compression (JPEG, MPEG).

3. **CMY/CMYK:**
   Used in color printing.
   Derived from RGB by subtracting each channel from 1.

4. **Lab Color Space:**
   Designed to be perceptually uniform.
   Useful in color correction and enhancement.

3. Conversion Example (RGB to HSV):

Hue (H): determines the type of color
Saturation (S): intensity of color
Value (V): brightness

MATLAB uses:

```
hsvImage = rgb2hsv(rgbImage);
```

4. Conversion Example (RGB to YCbCr):

```
ycbcrImage = rgb2ycbcr(rgbImage);
```

5. Advantages of Converting Color Spaces:

Simplifies tasks like detection and segmentation
Makes brightness-independent processing easy
Improves compression efficiency
Enhances color analysis and manipulation

Conclusion:

Converting RGB images to other color spaces is essential in image processing because different models separate color and intensity information in useful ways. These conversions enable more effective analysis, processing, and enhancement of color images.

4.7 The Basics of color image processing:

Color image processing involves techniques used to represent, manipulate, and analyze color images. Unlike grayscale images, which contain only intensity information, color images include multiple channels representing different color components.

1. Color Image Representation:

A color image is typically represented using **three separate color channels**.
Example:

RGB Model: Red, Green, and Blue components combine to form a full-color image.
A color pixel is represented as a vector:

$$[R, G, B]$$

2. Color Models (Color Spaces):

Different applications use different color models:

RGB: Used for display devices like monitors and cameras.
HSV (Hue, Saturation, Value): Separates color from brightness; useful for segmentation.
YCbCr: Used in video and image compression.
CMYK: Used in printing.

3. Color Transformations:

Changing between color spaces (e.g., RGB → HSV) helps in tasks like detection, enhancement, and filtering. MATLAB functions like `rgb2gray`, `rgb2hsv`, and `rgb2ycbcr` support such transformations.

4. Color Image Enhancement:

Operations such as contrast adjustment, histogram equalization of individual channels, and smoothing/sharpening are applied to improve visual quality.

5. Color Image Segmentation & Analysis:

Color information helps identify objects based on hue or saturation—for example, detecting skin color, traffic signs, or vegetation in images.

Conclusion:

Color image processing forms the foundation of digital image analysis. It uses color models, transformations, and enhancement techniques to represent and manipulate color information for applications in vision, graphics, multimedia, and pattern recognition.